

Solution Architecture Document

Sensei 1.0

Date completed: 7/25/2025

Last Updated: 7/29/2025

Document management history

Revision history

The following table summarizes the revision history for this document.				
Date	Version	Architect	Author/Title	Revision summary
07/25/2025	1.0	Elton Trotman	Solution Architecture	First Draft

Contents

Solution Architecture Document 1

 Document management history 1

 Introduction 4

 Executive summary..... 4

 Document Purpose 4

 Architectural goals..... 5

 Architectural Significant Requirements 7

 Architectural Patterns and Models 8

 Original Premise and Architecture 8

 Sensei Solution Architecture 9

 Sensei Architectural Solutions..... 10

 API Scaling Issues 10

 Workflow 10

 Event Consolidation and Logging..... 12

 Message Loss 12

 UI Response to Asynchronous Requests 12

 Architecture Scaling 13

 Key Patterns and Resources 17

 Service Bus Utility 17

 Dead Letter Queueing..... 18

Service Bus Asynchronous Processing -Asynchronous Request-Reply	19
Web Pub Sub	20
Async HTTP APIs (Durable Functions workflow)	21
Event Consolidation and Tracking.....	22
.....	22
Microservice Patterns	23
Overall Design.....	23
DDD	24
CRQS	25
Circuit breaker	26
Bulkhead	27
Saga.....	28
.....	28
Strangler Fig.....	29
Aggregation.....	30
Gateway Routing	31
Queue Based Load Leveling Pattern	31
Cross-cutting Components	32
Network Considerations.....	33
.....	33
Risks.....	35
DB Risk: Monolithic Schema Design	35
Risks for Microservices	36
Risk for Event Driven Systems	36
Architectural Compromises and Mitigation	Error! Bookmark not defined.
No Containerization	Error! Bookmark not defined.
No Gateway Offloading.....	Error! Bookmark not defined.
No Message Bridge.....	Error! Bookmark not defined.
No Backends for Frontends.....	Error! Bookmark not defined.
No Event Sourcing.....	Error! Bookmark not defined.
Testing	49

Stress Testing..... 50

Load Testing..... 51

Introduction

The goal of this Technical Architecture is to define the technologies, products, and services necessary to develop and support the system, and to ensure that the system components are compatible and comply with the enterprise-wide standards and services.

The objective of the Sensei program is to provide an architecture that achieves:

- high scalability both horizontally and vertically,
- event driven
- introduces microservices to the current monolithic architecture

The expected business outcomes include:

1. The current system will scale as demanded by SEAA requirements
2. The system can perform underload at the presentation, middle and data layers
3. Less reliance and significant cost reduction on the service provider (Azure) to achieve scalability and performance.
4. The architecture will be able to scale more easily.

Commented [A1]: Is this measurable in our context? We don't have a system under load for any period of time to be able to quantify this, but I suppose you can use MSFT cost estimator?

Commented [A2R1]: The current design offloads most of the cost on static web app and the function apps and places the load on service bus app plan which intended to be higher in compute.

Executive summary

Sensei is an architecture that allows for increased scaling and performance as the K12 program needs grow. With modern architectures, Sensei will be able to meet those needs.

Document Purpose

This document provides a comprehensive architectural overview of the system. It conveys the significant architectural decisions and focuses on providing:

- A description of the solution architecture, including major components and their interactions, processes, and data.
- A common understanding of the drivers (constraints and principles) influencing the architecture.
- Explanation of how the architecture satisfies the drivers.

Problem Domain

1. The original architecture proposed

- a. The architecture approved by CFI from Randstad was intended to resemble a microservice architecture complete with service bus, event grids and a domain driven design.
2. CFI stated existing application does not scale and fails under concurrency load
 - a. We were able to procure the following concurrency numbers from CFI:
 - i. A maximum of 80k concurrent users over an hour span.
 - ii. The time frame from max users is early March
3. Broad Performance issues

Architectural goals

1. Scalability
Beyond Vertical scaling is often necessary, introducing horizontal scaling provides benefit.
2. Always Secured
Limit access at presentation, business logic and data access layers. Use security controls at Web Apps, Web APIs, Microservices, SQL DB, BLOB Containers, to define security limitations.
3. Consolidation of events
Events from external webhooks, posts, routed via Event Grid/Service Bus should be centralized
4. Minimal Access
Each service has access to only those resources it needs at the level it needs.
one should "depend upon abstractions, [not] concretions."
5. Tracing, Logging and Auditing
Trace everything, log decisions, audit actions. Make it useful, traceable, and fast.
6. Network isolation
Each component of the solution will be isolated by VNets with Private end points
7. High Availability
The solution will be designed according to high availability principles.
8. Input Validation
Input validation will be performed at client and server sides.

9. Input will never have any unsecured content.

10. Asynchronous

Implement code asynchronously. No waiting or blocking.

11. Performance

The platform will be performant at the middle and data layers according to client requests

12. Downtime

Platform will not fail under load according to client's concurrency user requirements

13. Best Coding Practices

a. Simple & SOLID & DDD

Keep it simple, maintain single responsibility, and follow domain driven design principles.

Commented [A3]: Spell these out for our broad audience's sake

b. Single responsibility principle

"a class should have only a single responsibility (i.e. changes to only one part of the software's specification should be able to affect the specification of the class)."

Commented [A4]: Given how critical DDD is to the successful design of Microservices I think it warrants mention of use of a bounded context etc

c. Open/closed principle

"Software entities ... should be open for extension but closed for modification."

Commented [A5R4]: Robert and I need to review the schema for a true DDD break down, there's a common schema that currently makes this difficult.

d. Liskov substitution principle

"Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program."

e. Interface segregation principle

"Many client-specific interfaces are better than one general-purpose interface."

f. Dependency inversion principle

Architectural Significant Requirements

Non-Functional Requirements	
Concurrency	Platform will provide concurrent processing for Project data within the allotted SLA for data size of the engagement.
Logging	The platform will provide logging and tracing capabilities. The platform will provide log viewers with logged information.
Encryption	The platform will encrypt data at rest and at transfer. Platform will additionally encrypt C3+ data in metadata.
Reusability	Platform will support reusability across entire solution
Maintainability	Platform will keep maintainability
Flexibility	Platform will be extensible and flexible

Exception Handling	The platform will be catching all the exceptions across the system.
Scalability	The platform will have the capability to scale both Horizontally and Vertically
Performance	Platform will be performant within the stated parameters and requirements from the client
Concurrency	Platform will process multiple users simultaneously

Commented [A6]: Redundant entry

Architectural Patterns and Models

Original Premise and Architecture

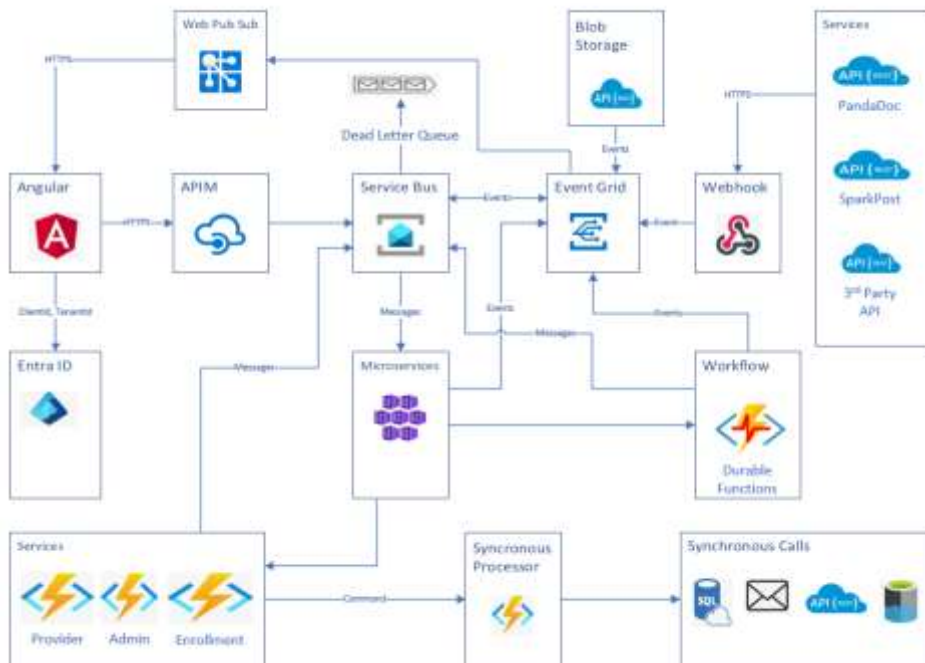
The Original Premise and Architecture is fairly widespread and addresses issues such as reliability, distribution, and coupling. The monolithic architecture currently in place does not address scaling, reliability, and flexibility at large.

Commented [A7]: The architecture is not a monolith, the codebase is.

Expanding upon the original architecture and augmenting the current architecture in place, Sensei looks to address:

- Asynchronous Processing
- Scaling Capabilities
- Message Reliability
- Event Consolidation
- UI Capability to Respond to Long Running and Asynchronous Requests

Sensei Solution Architecture



Commented [A8]: This is a rearranging of the same concepts on the last drawing, except in my drawing I'm identifying the bounded context that influences each distinct group of microservices.

Commented [A9R8]: Some key differences, there's the service lookup/invoke component (center bottom) for async processing, webhooks (upper right) that pandadoc and sparkpost utilize and the web Pub hub for the async response/reply pattern. This also keeps intact the current EntralD pattern without having to refactor much at all if any.

This architecture allows for a central messaging hub and event consolidation for applications via an Event Grid. This decouples the application from processing requests synchronously and defers synchronous processing to queue workers. The service bus allows for decoupling, reliable messaging, scalability and async communication. Microservices are a key architectural component, and sensei has the beginnings of a robust Microservice Pattern

Sensei Architectural Solutions

API Scaling Issues

Sensei offers options for handling compute/data intensive calls. From the client these calls can cause downtime in the application. The system crashed specifically with high demand.

Asynchronous Call Options

1. Sensei has addressed compute and/or data intensive synchronous bottle necks by establishing queues to offload synchronous calls. Messages are processed off queue and can be prioritized. This relies on the Asynchronous Reply-Reponse pattern as well as web PuB Sub. An alternative to web Pub Sub is the more antiquated https polling pattern that is tried and true yet less elegant and efficient.
 - a. Pros – Message Retry with dead letter queues, Priority Queueing, App Service Plan can change and scale
 - b. Cons - More complexity with Queueing
2. Sensei has also offered a pattern for retrieving responses to asynchronous API calls. Sensei applies the Asynchronous Reply-Response patterns for potentially intensive calls. APIs can be called asynchronously to achieve this.
 - a. Pros – Less Complexity
 - b. Cons - Message Loss Possible, No Priority Queueing, No Compute control, coupled to app service plan of App Service, no event tracking

Workflow

Sensei also offers options to handle long running functions:

1. Sensei utilizes durable functions to manage long running calls. There is built in support for workflow utilizing this message.
 - a. Pros – Less Complexity
 - b. Cons- Message Loss Possible, No Priority Queueing, No Compute control, coupled to app service plan of App Service
2. Sensei's usage of queueing and the Asynchronous Reply-Response pattern addresses long running calls.
 - a. Pros – Message Retry, Priority Queueing, App Service Plan can change and scale, event tracking
 - b. Cons- More complexity with Queueing

Durable Functions Workflow

In Azure Durable Functions, a workflow defines a sequence of operations that can involve multiple steps, long-running tasks, and interactions with external systems. Unlike standard, stateless Azure Functions, Durable Functions introduce stateful execution, allowing workflows to pause, resume, and maintain their state across executions.

Key components that define a workflow in Azure Durable Functions:

Durable Task Framework:

This underlying framework manages the state and execution history of orchestrations in durable storage. It enables the "checkpointing" of workflow progress, allowing orchestrations to be resilient to failures and resume from where they left off.

Orchestrator Functions:

They define the logical flow and coordination of tasks. Orchestrator functions must be deterministic, meaning they always produce the same output for a given input, ensuring reliable replay and state management. They use a specific API to schedule activity functions, manage state, and handle external events.

Activity Functions:

These functions perform the actual work within the workflow. They are typically stateless and carry out discrete units of work, such as making API calls, interacting with databases, or processing data. Orchestrator functions invoke activity functions, and the Durable Functions runtime manages the execution and retries of these activities.

Client Functions:

These functions initiate the execution of an orchestrator function. They act as the entry point for the workflow, often triggered by external events like HTTP requests, queue messages, or timers. Client functions use the Durable Functions client API to start, query, and manage orchestration instances.

Event Consolidation and Logging

With Event Grid usage, Sensei not only delivers event-based architecture, but also the ability to consolidate events, be they 3rd Party (webhooks), custom application events or Azure triggers.

Message Loss

Sensei Offers dead letter queueing to retry messages that were unable to be processed. This is especially critical when under high load

UI Response to Asynchronous Requests

A combination of Web Pub Sub and the Asynchronous Response-Reply Pattern allows for the UI to respond to long running/compute intensive processes.

Implementing an Asynchronous Request-Reply pattern with Azure Web PubSub and an Angular client allows you to manage long-running server-side tasks without blocking the client's user interface. The Angular application sends a request, and a backend service handles the work, using Web PubSub to push the result back to the specific client connection when it's ready.

Architecture overview

3 main components:

1. Angular: Initiates a request and uses the web-pubsub-client library to maintain a persistent WebSocket connection to Web PubSub, where it listens for a specific reply.
2. Azure Web PubSub: A managed service that handles the WebSocket connections for you. It relays messages between the client and the backend application.

Commented [A10]: Maybe tighten this up a little. Event Grid is not as resilient as the ServiceBus queues so 3rd party webhooks that require delivery guarantees could be a bad fit for this service. Also, I don't see any mention of EventHub, these solve unique use cases (push v pull etc) which is why I included them in the tech stack for the OG service bus.

Commented [A11R10]: Webhooks typically don't generate much traffic so eventhubs may be overkill here. Let's check the use case other than signing confirmation of documents.

3. Backend: code receives the initial request, starts the long-running task, and uses the web-pubsub server-side SDK to send the reply message back to the client.

Order of events

1. Client request: The Angular component calls `webPubSubService.sendRequest()`.
2. Message to Web PubSub: The service client sends a message containing a `correlationId` to the `async-replies` group on the Web PubSub service.
3. WebHook trigger: Web PubSub, configured with an event handler, forwards the client message to your backend application's `/api/message` endpoint.
4. Backend processing: The backend server receives the message, extracts the `correlationId` and `connectionId`, and starts the long-running task. It immediately sends an HTTP 202 status back to Web PubSub to confirm receipt.
5. Task completion and reply: Once the backend task is complete, it uses the server-side SDK to send the result, along with the original `correlationId`, back to the specific `connectionId` on Web PubSub.
6. Reply delivery: Web PubSub forwards the reply message to the Angular client over the open WebSocket connection.
7. Client receives reply: The `web-pubsub.service`'s message handler receives the reply. The RxJS filter operator ensures that only the message with the matching `correlationId` is processed, and the component's `subscribe()` block is triggered with the final result.

Architecture Scaling

Sensei has established a lightweight microservices pattern over monolithic to achieve horizontal scaling. Sensei will continually add more complex microservices pattern as the architecture matures. Currently sensei accounts for the core patterns to achieve a microservices architecture.

Resiliency

Dapr achieves failures and retries through its resiliency building block, which is configured declaratively using a YAML file. This approach decouples the resilience logic from code. Dapr applies these policies to transient failures that occur during communication between its sidecar and your application or other infrastructure components.

Core components of Dapr resiliency

- **Resiliency spec:** A YAML file, often named `resiliency.yaml`, where you define the specific policies for handling transient failures.
- **Timeouts:** Your first line of defense, timeouts prevent applications from hanging indefinitely while waiting for an unresponsive service. A request that exceeds the timeout duration will trigger the retry policy.
- **Retries:** If an operation fails, Dapr automatically reattempts the operation based on a configured policy. Retries are especially useful for handling temporary issues like network glitches or a service being temporarily unavailable.
- **Back-off strategies:** Dapr supports two main back-off strategies for retries to avoid overwhelming a struggling service with repeated requests.
 - **Exponential back-off:** Increases the delay between retry attempts exponentially. Dapr also adds jitter (random variation) to the delay to prevent a "retry storm" where multiple clients hit a recovering service simultaneously.
 - **Constant back-off:** Retries at a fixed interval.
- **Circuit breakers:** Inspired by the electrical component, this pattern prevents a failing service from causing cascading failures. After a configurable number of failures, the circuit "trips" and all subsequent requests fail immediately without even attempting to reach the unhealthy service. This gives the failing service time to recover.

Retries in specific Dapr building blocks

Service invocation

- **Mechanism:** When one Dapr sidecar makes a service-to-service call to another, Dapr's built-in logic automatically handles retries for failures like network connectivity issues or sidecar availability.
- **Configuration:** You can define custom retry policies in your resiliency spec and apply them to specific application targets to override the default behavior.

Pub/Sub

- **Mechanism:** Dapr uses a combination of its own resiliency policies and the message broker's built-in retry mechanisms.
- **Dead letter queues:** With supported components like Azure Service Bus, you can configure Dapr to send messages to a dead-letter queue after a certain number of failed processing attempts. This prevents "poison messages" from causing infinite retries.

Actors

- **Mechanism:** Dapr offers built-in retry logic for failures during the sidecar-to-sidecar communication for actor method calls. For durable actor state and reminders, retries are also built-in.
- **Runtime exceptions:** If an actor method throws an exception, the Dapr runtime logs the error but does not automatically retry the actor's method itself. For true durability, you must use persistent state and reminders to re-trigger the logic.

State management

- **Mechanism:** Dapr automatically retries requests to state stores, like saving or retrieving data.

Interservice Communication

Depending on the specific requirements for synchronous or asynchronous communication and the level of coupling desired between services.

Service Mesh (e.g., Dapr, Istio on AKS):

Dapr (Distributed Application Runtime) provides a dedicated infrastructure layer for handling interservice communication. Sidecar proxies are deployed alongside each microservice to manage communication.

Synchronous Communication:

[HTTP/REST:](#)

Microservices can directly communicate with each other using HTTP requests, often with RESTful APIs. This is suitable for scenarios where an immediate response is required. Azure Container Apps, for instance, facilitates this by providing internal DNS resolution for services within the same environment

Asynchronous Communication

Azure Service Bus:

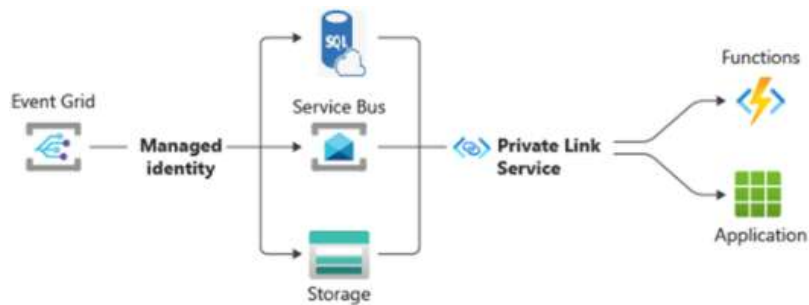
This is a robust messaging service for reliable, asynchronous communication. It supports queues for point-to-point messaging and topics/subscriptions for publish-subscribe patterns, enabling decoupling between services.

Azure Event Grid:

A fully managed event routing service that allows services to react to events published by other services or Azure resources. It's ideal for event-driven architectures where services need to be notified of changes or actions

Key Patterns and Resources

Service Bus Utility



Azure Service Bus is a fully managed enterprise message broker with message queues and publish-subscribe topics.

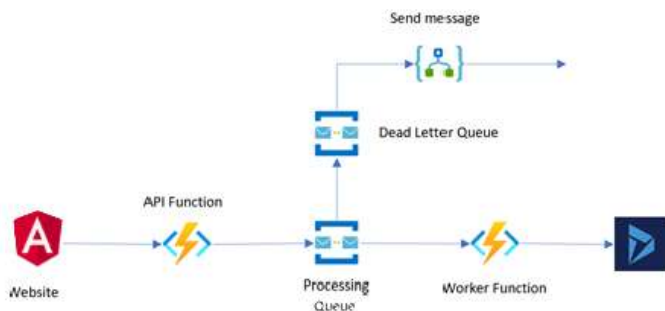
- Load-balancing work across competing workers
- Safely routing and transferring data and control across service and application boundaries
- Coordinating transactional work that requires a high degree of reliability
- Data is transferred between different applications and services using **messages**.

Some common messaging scenarios are:

- **Decouple applications.** Improve reliability and scalability of applications and services.
- **Transactions.** Allows you to do several operations, all in the scope of an atomic transaction. For example, the following operations can be done in the scope of a transaction.

<https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-overview>

Dead Letter Queueing

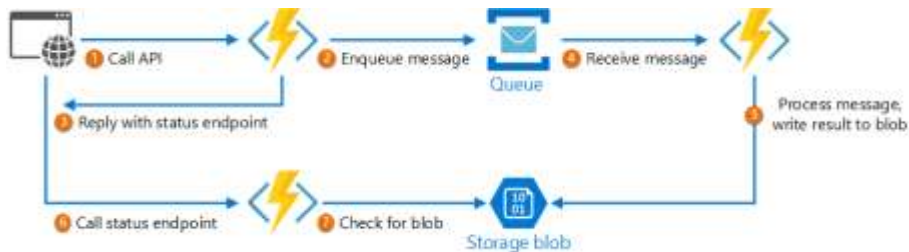


The purpose of the dead-letter queue is to hold messages that can't be delivered to any receiver, or messages that couldn't be processed. Messages can then be removed from the DLQ and inspected. An application might let a user correct issues and resubmit the message.

From an API and protocol perspective, the DLQ is mostly similar to any other queue, except that messages can only be submitted via the dead-letter operation of the parent entity. In addition, time-to-live isn't observed, and you can't dead-letter a message from a DLQ. The dead-letter queue fully supports normal operations such as peek-lock delivery, receive-and-delete, and transactional operations.

<https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-dead-letter-queues>

Service Bus Asynchronous Processing -Asynchronous Request-Reply



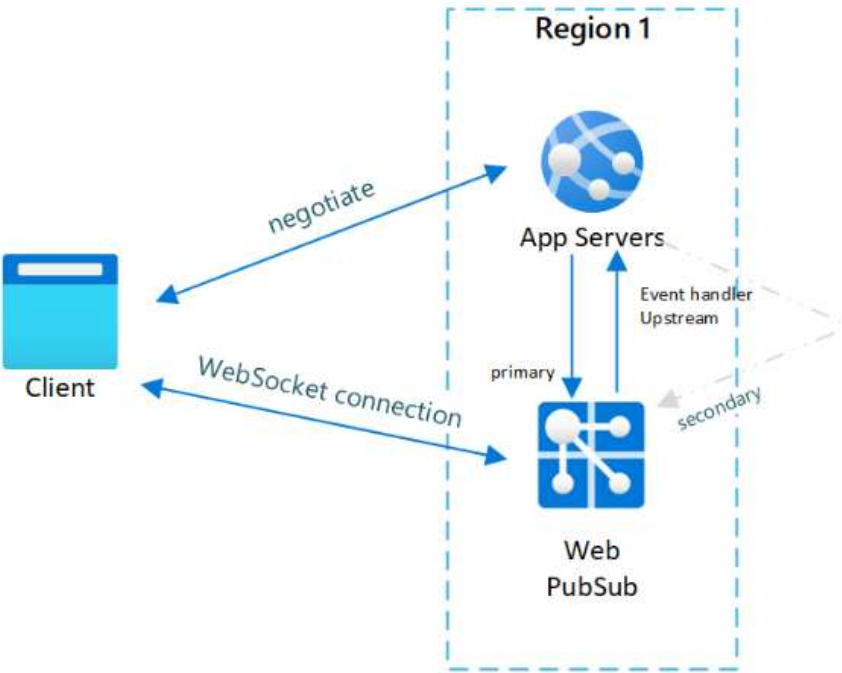
Change synchronous API calls to an asynchronous pattern using Azure Service Bus. This involves decoupling the request initiation from the response delivery. This allows the client to receive an immediate acknowledgment while the actual processing occurs in the background, mediated by Service Bus.

Steps to achieve this:

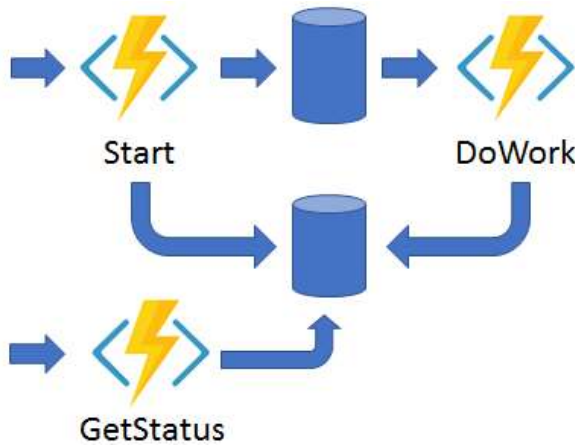
- Client initiates requests and receives immediate acknowledgment:
- The client sends its request to an API endpoint (e.g., a Web API or Azure Function).
- Instead of performing the full processing synchronously, this API immediately publishes a message to an Azure Service Bus queue or topic, containing the request details.
- The API then returns an immediate response to the client (e.g., HTTP 202 Accepted), indicating that the request has been received and will be processed asynchronously.
- A separate worker process or Azure Function subscribes to the Service Bus queue/topic and consumes the messages.
- This consumer performs the actual business logic and processing of the request.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/async-request-reply>

Web Pub Sub



Async HTTP APIs (Durable Functions workflow)

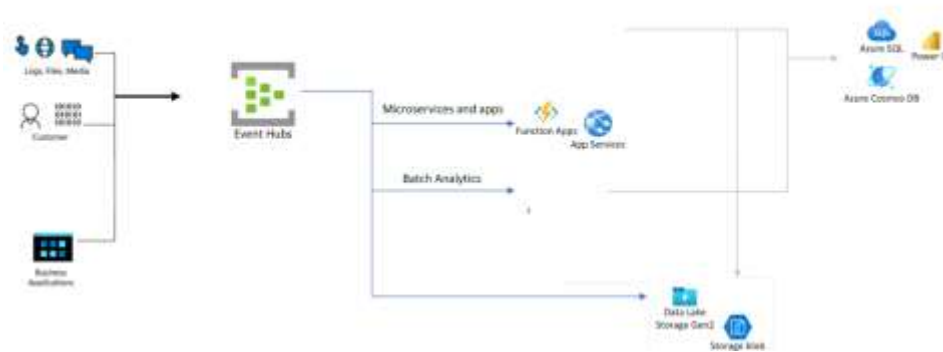


“The async HTTP API pattern addresses the problem of coordinating the state of long-running operations with external clients. A common way to implement this pattern is by having an HTTP endpoint trigger the long-running action. Then, redirect the client to a status endpoint that the client polls to learn when the operation is finished.

Durable Functions provides *built-in support* for this pattern, simplifying or even removing the code you need to write to interact with long-running function executions. For example, the Durable Functions quickstart samples ([C#](#), [JavaScript](#), [TypeScript](#), [Python](#), [PowerShell](#), and [Java](#)) show a simple REST command that you can use to start new orchestrator function instances. After an instance starts, the extension exposes webhook HTTP APIs that query the orchestrator function status.”

<https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview?tabs=in-process%2Cnodejs-v3%2Cv1-model&pivots=csharp>

Event Consolidation and Tracking



An event is created by a publisher such as a Blob Storage account, **Event Hubs** or even an Azure subscription. As events occur, they're published to an endpoint called a topic that the Event Grid service manages to digest all incoming messages.

If they can post an HTTP request to the Event Grid service, then they're candidates for sending events.

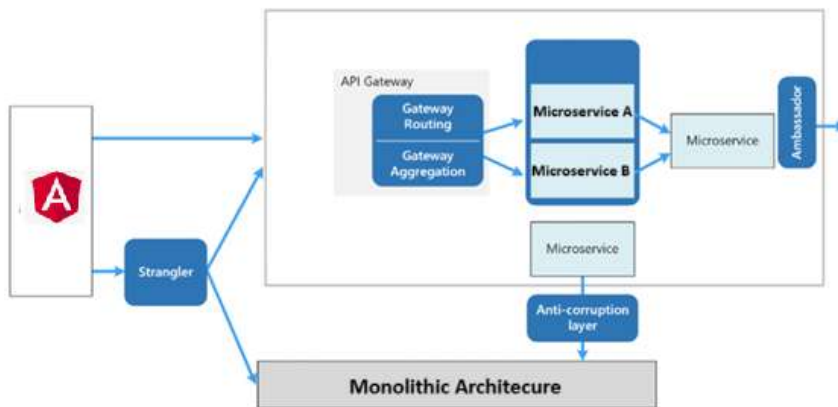
These highlight some of the emerging serverless technologies on Azure, such as Functions and Logic Apps. In addition to Azure Automation, another type of event handler could be any HTTP callback, also referred to as a Webhook.

<https://learn.microsoft.com/en-us/archive/msdn-magazine/2018/february/azure-event-driven-architecture-in-the-cloud-with-azure-event-grid>

Commented [A12]: Aha! Found 'em

Microservice Patterns

Overall Design



Microservices is an architectural style that structures an application as a collection of small, independent services. Each service focuses on a specific business capability and communicates with others through well-defined APIs, enabling independent development, deployment, and scaling.

- **Independent Services:**

Each microservice is a self-contained unit responsible for a specific function or business capability.

- **Lightweight Communication:**

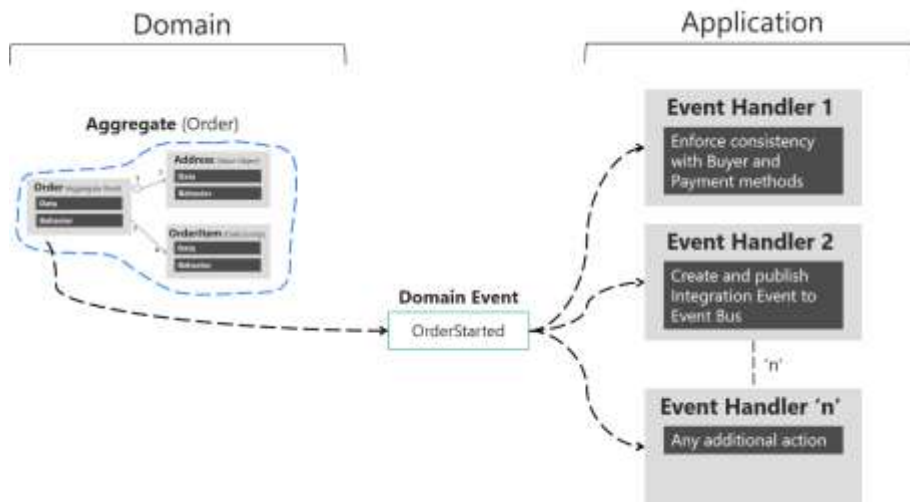
Services communicate with each other through APIs, often using protocols like REST.

- **Fault Isolation:**

If one service fails, it doesn't necessarily bring down the entire application, improving overall resilience.

<https://learn.microsoft.com/en-us/azure/architecture/microservices/design/patterns>

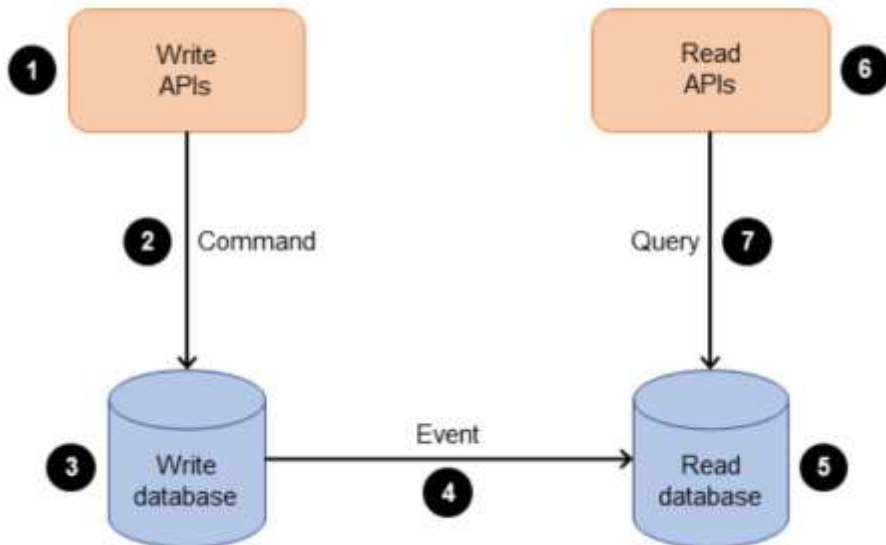
DDD



DDD is a software development approach that centers the design and development of a software system around the core business domain. It emphasizes understanding the business domain deeply and modeling the software to accurately reflect its complexities and nuances. This approach aims to create a shared understanding of the domain among all stakeholders (business experts, developers, etc.) using a [ubiquitous language](#).

<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/ddd-oriented-microservice>

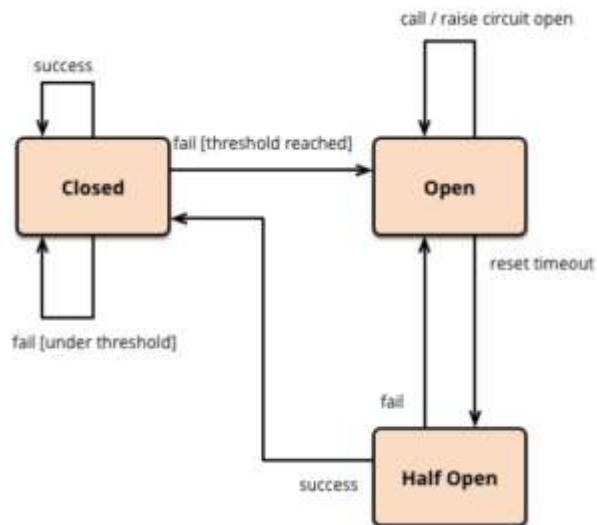
CRQS



1. The business interacts with the application by sending commands through an API. Commands are actions such as creating, updating, or deleting data.
2. The application processes the incoming command on the command side. This involves validating, authorizing, and running the operation.
3. The application persists the command's data in the write (command) database.
4. After the command is stored in the write database, events are triggered to update the data in the read (query) database.
5. The read (query) database processes and persists the data. Read databases are designed to be optimized for specific query requirements.
6. The business interacts with read APIs to send queries to the query side of the application.
7. The application processes the incoming query on the query side and retrieves the data from the read database.

<https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-data-persistence/cqrs-pattern.html>

Circuit breaker

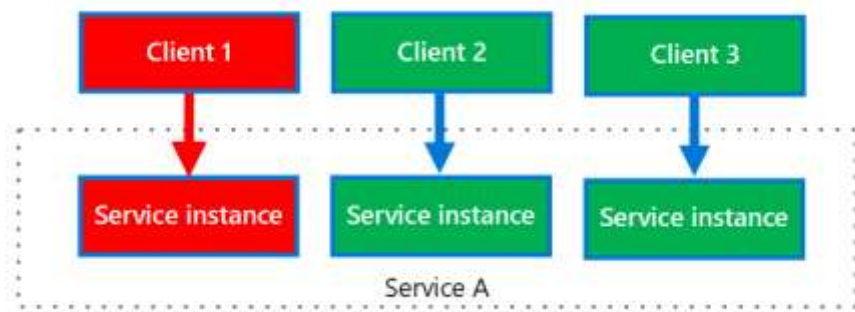


It is used to prevent cascading failures. It acts as a proxy, monitoring the success and failure of operations, and temporarily blocking further requests to a failing service once a predefined threshold of failures is reached. This prevents the entire system from being overwhelmed by a failing dependency.

- The Circuit Breaker wraps a potentially failing operation (like a remote service call).
- It monitors the success and failure of these calls.
- When failures exceed a certain threshold, the circuit "trips," blocking further calls to the failing service.
- After a timeout period, the circuit "half-opens," allowing a limited number of requests to test if the service has recovered.
- If the tests succeed, the circuit closes, and normal operation resumes.
- If the tests fail, the circuit remains open, and the process repeats.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>

Bulkhead



The Bulkhead Pattern is a design pattern used to improve fault tolerance in distributed systems, particularly in microservices architectures. It isolates components or services, preventing failures in one part of the system from cascading and causing widespread outages. This isolation is achieved by limiting the resources allocated to each component, such as thread pools or database connections.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/bulkhead>

Saga

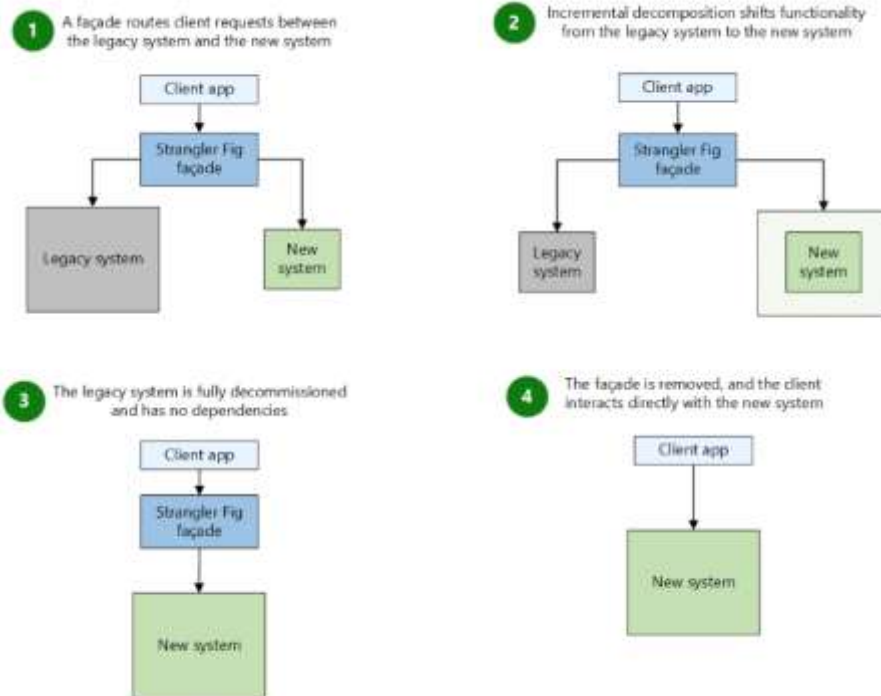


The saga pattern is a design pattern used in microservices architecture to manage data consistency across multiple services during distributed transactions. It involves a sequence of local transactions, where each transaction updates the database and potentially triggers the next one in the sequence. If a local transaction fails, compensating transactions are executed to undo the changes made by previous successful transactions, ensuring eventual consistency.

- Local Transactions:
Each service in the saga performs its own local transaction, updating its own database.
- Eventual Consistency
Unlike traditional ACID transactions, data inconsistencies might exist temporarily during the saga execution, but they are eventually resolved.
- Compensating Transactions
If a local transaction fails, compensating transactions are executed to roll back the changes made by the successful transactions before it.
- Orchestration
A central coordinator (orchestrator) manages the saga flow, sending commands to each service and receiving responses.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/saga>

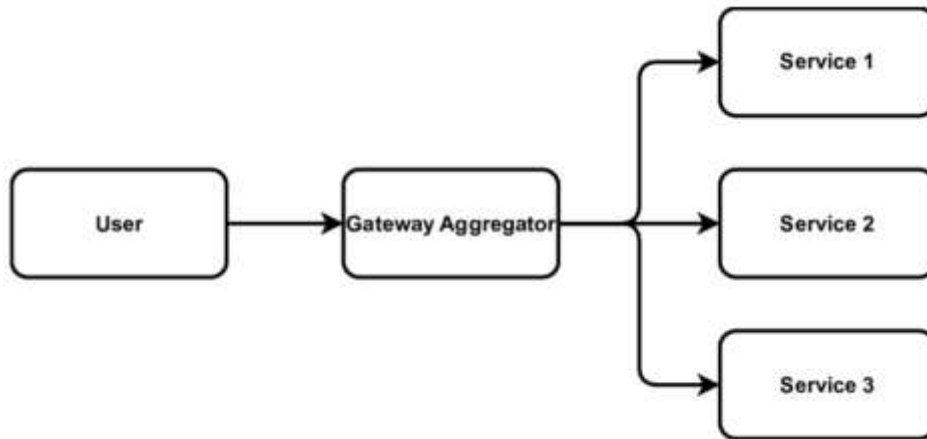
Strangler Fig



This pattern incrementally migrates a legacy system by gradually replacing specific pieces of functionality with new applications and services. As you replace features from the legacy system, the new system eventually comprises all of the old system's features. This approach suppresses the old system so that you can decommission it.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>

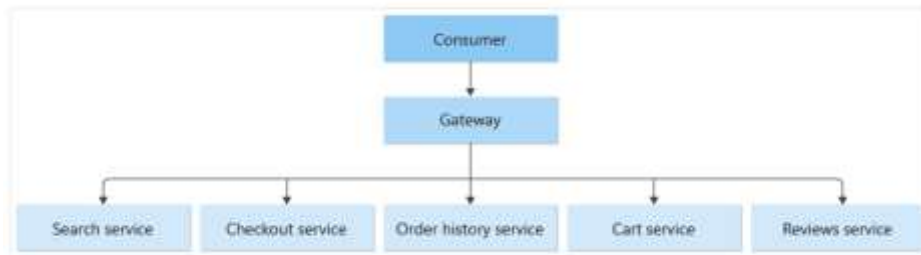
Aggregation



To perform a single task, a client may have to make multiple calls to various backend services. An application that relies on many services to perform a task must expend resources on each request. When any new feature or service is added to the application, additional requests are needed, further increasing resource requirements and network calls. This chattiness between a client and a backend can adversely impact the performance and scale of the application. Microservice architectures have made this problem more common, as applications built around many smaller services naturally have a higher amount of cross-service calls.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-aggregation>

Gateway Routing



When a client needs to consume multiple services, multiple service instances or a combination of both, the client must be updated when services are added or removed. Consider the following scenarios.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-routing>

Queue Based Load Leveling Pattern

“Use a queue that acts as a buffer between a task and a service it invokes in order to smooth intermittent heavy loads that can cause the service to fail or the task to time out.

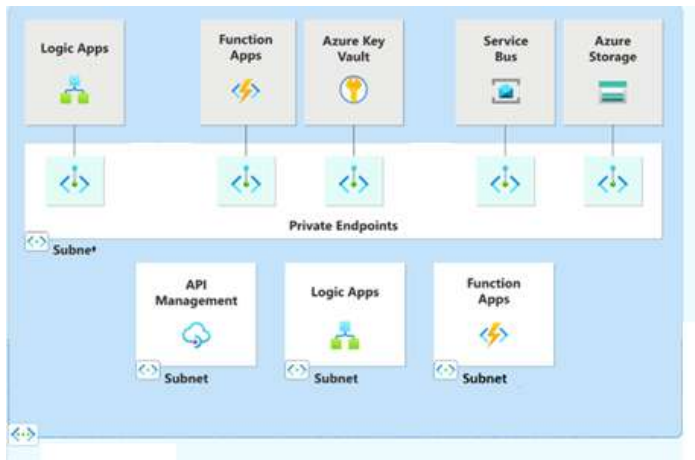
This can help to minimize the impact of peaks in demand on availability and responsiveness for both the task and the service.”

<https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling>

Cross-cutting Components

1. Validation: classes and attributes to handle validation easily.
2. Gate Checking: classes and attributes to handle comparison of expected results vs actual.
3. Domain Model: DDD base classes for app services, domain services, domain repositories, entities, specifications, and composite roots.
4. Encryption: classes to perform encryption/decryption according to Platform and App use cases.
5. Configuration: classes providing configuration infrastructure and change detection.
6. Jobs: orchestrator, implementation base classes, job monitoring, status tracking and retrying classes.
7. Business Domain Capabilities: set of project libraries created to utilize specific domain capabilities of the platform. Including service principal management.
8. Logging, Tracing, Auditing and Exception handling: base classes providing those capabilities.

Network Considerations



A private endpoint is a network interface that uses a private IP address from your virtual network. This network interface connects you privately and securely to a service that's powered by Azure Private Link. By enabling a private endpoint, you're bringing the service into your virtual network.

- Platform will ensure network level security by creating resources in designated VNets or limiting IP firewall to well-known list of IP addresses.
- Microservices and Azure functions will be connected to platform Public Endpoints.
- Blob storage and Data Lake Store will be connected to platform Public Endpoints.
- Sensei consists of microservices and an event driven design. Sensei will be deployed in supported geographical regions.

Security

In Azure, security for microservices is based on the Zero Trust principle, using a multi-layered, defense-in-depth approach. This strategy relies on native Azure services to manage identity, secure communication, protect data, and monitor the entire environment.

Foundational strategies

- **Implement Zero Trust:** Assume that no microservice, user, or device is trustworthy by default. Every transaction must be explicitly verified and authenticated.

Identity and access management

Securing access for both human users and microservice identities is critical in a distributed system.

- **Managed identities:** Assign managed identities to microservices to authenticate with other Azure resources, like Key Vault or databases, without needing to manage credentials manually.
- **Role-Based Access Control (RBAC):** Define the principle of least privilege by using RBAC. Grant each identity only the permissions required to perform its function.
- **Use tokens for authentication:** For service-to-service communication, use OAuth 2.0 and OpenID Connect (OIDC) with JSON Web Tokens (JWTs) for secure and standardized authentication.

Secure communication

- **API Gateway:** Use Azure API Management as the single entry point for all client requests
- **Encrypt data in transit:** Enforce Transport Layer Security (TLS) or mutual TLS (mTLS) for all communications.
 - **External traffic:** Enforce HTTPS for all external API endpoints.
 - **Internal traffic:** Implement mTLS to authenticate and encrypt traffic between microservices. Dapr can automate mTLS.
- **Segment networks:** Use Azure virtual networks (VNETs) to isolate microservices from the public internet.
 - **Network Security Groups (NSGs):** Control network traffic by creating NSG rules that allow only necessary traffic between services and subnets.
 - **Private Link:** Securely access Azure PaaS services like Azure Key Vault and Azure Cosmos DB from within your private network.
- **Protect against DDoS attacks:** Enable Azure DDoS Protection to defend

Data protection

Securing sensitive data, both at rest and in motion, is a top priority.

- **Store secrets securely:** Use Key Vaults
- **Encrypt data at rest:** Ensure data stored in databases, storage accounts, and file systems is encrypted
- **Use database-level isolation:** Implement a dedicated database for each microservice to prevent a breach in one service from compromising data belonging to another.

Monitoring and threat detection

Microservice architectures require a centralized approach to monitoring and auditing to manage complexity.

- **Centralize logs and metrics:** Aggregate logs from all microservices into a central location to Log Analytics.
- **Use Microsoft Defender for Cloud:** Manages security and provides cloud workload protection, including recommendations and threat detection for containers.
- **Monitor container security:** Actively scan container images for vulnerabilities in your Azure Container Registry before they are deployed.

Code and dependency security

Address security risks in the application code and its dependencies.

- **Scan for vulnerable dependencies:** Use Sonar Cube to continuously scan your application's source code and third-party libraries for security vulnerabilities.
- **Harden container images:** Use minimal, trusted base images and remove unnecessary components to reduce the attack surface

Risks

DB Risk: Monolithic Schema Design

The current monolithic schema is highly dependent on a common schema. This schema will need to be incrementally denormalized with each new domain. It was suggested that

we can use logical domains. Once a working pattern with logical domains is created, we can in the future move to separate databases.

Risks for Microservices

- **Larger Attack Surface:**
More services and communication points mean more potential entry points for attackers, increasing the risk of unauthorized access, data breaches, and denial-of-service attacks.
Implementing consistent security policies across multiple services with varying configurations and data flows can be challenging.
- **Increased Complexity:**
Managing and maintaining a distributed system with numerous independent services can be more complex than managing a single monolithic application.
- **Data Fragmentation:**
Data can become fragmented across different services, making it harder to manage and analyze.

Commented [A13]: Is this much of a risk in a shared nothing & zero trust architecture?

Commented [A14R13]: I'd say the implementation of private end points factors greatly

Risk for Event Driven Systems

- **Debugging**
The asynchronous nature of Event Driven Architectures makes debugging difficult. Tracing event flows and identifying the root cause of issues can be challenging.
- **Idempotency**
Failing to handle idempotency can lead to duplicate event processing and inconsistent states.
- **Error Handling**
Handling errors and exceptions in an Event Driven Architectures can be more complex due to the asynchronous nature of events.
- **Testing**
Testing Event Driven Architectures systems can be challenging due to the potential for unforeseen interactions and permutations.

Developer Tooling & Framework Standards

1. Purpose

Establish a consistent, auditable, and high-quality developer experience for all K12 Sensei microservices. This document codifies the minimum required tooling, project structure, testing layers, local development workflow, and quality/security gates so that new services are production-ready by construction.

2. Scope

- Service scaffolding templates and repository layout
- Supported runtimes, frameworks, and base container images
- Local development workflow (Dapr, emulators, environment config)
- TBD
 - Integrate/replace powershell orchestration(s) with Aspire.net
 - Generate deployment manifests (artifacts) with Aspire for target environments
- Tooling catalog (build, test, quality, security, observability)
- Mandatory quality gates and coverage thresholds (TBD)
- Test strategy (unit, contract, integration, performance smoke)
- Secret handling and configuration hygiene
- Governance hooks (pre-commit, CI/CD gates) (TBD)

3. Goals & Success Metrics

Goal	Metric / Target
Fast Onboarding	New engineer productive (PR merged) ≤ 60 min
Consistent Quality	Sonar "no new critical/blocker issues" enforced
Early Security	0 committed secrets (Gitleaks)
Reliable Contracts	Contract test failures block merge 100%
Observability Baseline	100% services emit RED metrics + tracing upon first run

5. Supported Language & Runtime Matrix

Layer	Primary	Policy	Notes
Backend APIs & Workers	.NET 8 (LTS)	Mandatory	Evaluate Native AOT (Phase 2 performance)
Workflow / Orchestration	Durable Functions (.NET 8) + Dapr Workflow (eval)	Allowed	ADR required before switching primary
Front-end	Angular (current LTS)	Mandatory	Node version pinned via .nvmrc
Scripting / Infra	PowerShell, Bash, Terraform	Mandatory	Terraform authoritative for infra
Container Runtime Base	mcr.microsoft.com/dotnet/aspnet:8.0-alpine	Mandatory	Weekly digest refresh PR

6. Standard Service Template

A k12-cli (internal) or dotnet new k12.api generator produces:

```

/service-root
  /src
    ServiceName.Api
    ServiceName.Application
    ServiceName.Domain
    ServiceName.Infrastructure
    ServiceName.Contracts (DTOs + events + schema JSON)
  /tests
    ServiceName.UnitTests
    ServiceName.ContractTests
    ServiceName.IntegrationTests
    ServiceName.PerformanceSmoke
  /build
    Dockerfile
    sonar-project.properties
  /ops
    terraform/
    deployment/ (ACA manifests or placeholders)
  /docs
    adr/
    sequences/
  Directory.Build.props
  global.json

```

Layer Responsibilities:

- Domain: Pure domain model + invariants (no external references)
- Application: Use cases (commands/queries), orchestration logic
- Infrastructure: Persistence (EF Core or repository), messaging adapters, external API clients
- API: Transport concerns (controllers, validation, mapping)
- Contracts: Message definitions, schema JSON used in validation & contract tests

7. Project Conventions

Concern	Convention	Rationale
Namespaces	Company.{Project}.{BoundedContext}.{Layer}	Consistent resolution
DTO Mapping	Mapster or Manual Profiles (no reflection automagic)	Performance & explicitness

Validation	FluentValidation	Uniform error shape
Configuration	Options + validation on startup	Fail fast
Health Endpoints	/health/live & /health/ready	Platform probes
OpenAPI	Swashbuckle w/ minimal docs	Contract visibility
Error Shape	ProblemDetails (RFC 7807)	Standardization
Logging	Structured (Serilog or built-in + OTel)	Queryable
Tracing	OpenTelemetry (W3C traceparent)	Cross-service correlation
Messaging	Dapr pub/sub abstraction	Decouple from direct SDK
Integrity	integrityHash on envelope	Tamper detection

8. Quality Gates (Local & CI)

Layer	Tool	Threshold / Rule	CI Enforcement
Formatting (.NET)	dotnet format	No diffs	Yes (fail)
Lint (Angular)	ESLint + Prettier	No errors	Yes
Static Analysis	Roslyn + SonarQube	No new critical/blocker	Yes
Code Coverage	Coverlet	≥ 70% lines (raise to 80% Phase 2)	Yes
Vulnerability Scan	Trivy	0 Critical; High requires waiver	Yes
Secrets	Gitleaks	0 findings	Yes
Contract Tests	Pact + schema validation	All passing	Yes
Build Reproducibility	Deterministic flags	Hash stable	Yes (hash diff check)

Idempotency Compliance	Analyzer (custom)	Mandatory attribute where required	Yes (warn→fail Phase 2)
---------------------------	-------------------	---------------------------------------	----------------------------

9. Testing Strategy

Layer	Purpose	Scope	Trigger	Notes
Unit	Domain integrity	Aggregates, value objects	Every build	<100ms/test ideal
Contract (REST)	Provider-consumer alignment	Pact interactions	PR + nightly	Fails merge if drift
Contract (Events)	Envelope shape & schema	JSON schema validation	PR + nightly	AsyncAPI future
Integration	Infra boundaries	DB, message bus pub/sub	Selective on PR	Full nightly
Performance Smoke	Regression guard	5–10 scripted endpoints	Pre-prod deploy	k6 minimal load
Security Static	Dependencies & secrets	FS & image scan	Every build	Trivy + Gitleaks
Resiliency (Phase 2)	Failure handling	Chaos inject tests	Quarterly	Chaos Studio/Dapr faults

Coverage Reporting:

- Aggregated via ReportGenerator (lcov + HTML)
- Published as pipeline artifact
- Trend tracked in Quality dashboard

10. Local Development Workflow

Golden path:

1. Clone repository
2. Run ./bootstrap.(ps1|sh) (installs SDK versions, Dapr, commit hooks)
3. Start dependencies: dapr run or docker compose up
4. Launch service: dapr run --app-id awarding-api -- dotnet run --project src/Awarding.Api

- 5. Execute tests: make test or dotnet test
- 6. View coverage: make coverage
- 7. Build container: make image
- 8. Run Trivy scan: make scan

10. Secret & Configuration Handling

Concern	Dev Approach	Production Approach	Rule
App Secrets	dotnet user-secrets or .env (gitignored)	Key Vault + Managed Identity	Never in repo
Connection Strings	Env var override	Key Vault reference	Must support rotation
API Keys	Rotated dev keys (low-priv)	Managed Identity preferred	Expire ≤ 90 days
Certificates	Self-signed dev trust	Issued + rotated	No wildcard in prod
Feature Flags	Env bools (temp)	Central config (App Config future)	Document toggles

11. Dapr (Local Minimal Set)

Capability	Component	Purpose
Pub/Sub	Azure Service Bus (dev namespace) or in-memory fallback	-
State Store	Redis (container)	Ephemeral counters / throttling
Secrets	Key Vault binding	-
Tracing	OTel exporter	-

12. Tooling Matrix (Authoritative Extract)

Category	Tool	Mandatory	Version Policy	Owner
Build	.NET SDK 8	Yes	Global.json pinned	Platform

Build Orchestration	Make / PowerShell scripts	Yes	Script repo	Platform
Container Build	Docker BuildKit	Yes	≤ 2 minors behind	DevOps
Pub/Sub Abstraction	Dapr CLI	Yes	Latest stable	Platform
Static Analysis	Roslyn + SonarQube	Yes	Quality profile locked	Architecture
Contract Testing	Pact	Yes	Broker pinned	QA
Message Schema Validation	Custom schema validator (JSON)	Yes	Synchronized	Architecture
Security Scan	Trivy	Yes	Weekly update	Security
Secret Scan	Gitleaks	Yes	Monthly update	Security
SBOM	Syft	Yes	Latest minor	Security
Coverage	Coverlet	Yes	Locked minor	Dev Leads
Load (Smoke)	k6	Conditional	LTS binary	Performance
Observability	OpenTelemetry SDK + Collector	Yes	Pinned minor	Platform

13. Command Cheat Sheet (for local & CI/CD cmds)

Task	Command
New Service	K12-cli new service --name Awarding
Restore & Build	dotnet build
Run API	dapr run --app-id awarding-api -- dotnet run --project src/Awarding.Api
All Tests	make test
Contract Tests	dotnet test tests/Awarding.ContractTests
Format Code	make format

Generate Coverage	make coverage
Build Image	docker build -t k12/awarding-api:dev .
Scan Image	trivy image k12/awarding-api:dev
Generate SBOM	syft k12/awarding-api:dev -o spdx-json > sbom.json
Run Lint (Angular)	npm run lint
Publish Pact	npm run pact:publish or .NET pact CLI

14. Governance & Automation Hooks

Stage	Hook	Enforcement
Pre-Commit	Format + lint + fast secret scan	Developer machine
Push (PR)	Build + unit + contract + coverage	Mandatory
PR Gate	SonarQube quality gate + Trivy scan	Must pass
Merge to Main	Full integration tests + image publish	Blocking failures
Promotion	Synthetic tests + manual approval (initial)	Terraform plan reviewed (TBD)

15. Observability Requirements (Per Service)

Requirement	Description
RED Metrics	request_rate, request_error_rate, request_duration_p95
Domain Metrics	awarding_batches_completed_total (if domain relevant)
Tracing	100% non-prod, 20% head + tail in prod
Logging	Structured + correlationId + traceId
Integrity Metrics	integrity_hash_mismatch_count exported

16. Idempotency & Integrity Enforcement

- Idempotency required for every message consumer (duplicate detection data store).
- integrityHash computed: canonical JSON payload (sorted keys) + messageId → SHA-256 hex (prefixed sha256:).
- Analyzer (Phase 1) ensures Consumer handlers are Idempotent; violations reported.

17. Security Controls (Developer Workflow)

Control	Mechanism	Failure Handling
Secret Leakage	Gitleaks	Pipeline fail
Vulnerabilities	Trivy	Fail on Critical
Supply Chain (Phase 2)	Image signing	Block unsigned
Authorization Checks	Integration tests w/ sample tokens	Test failure
PII Logging Guard	Structured logging filters	Failing test if pattern matched

18. Onboarding Golden Path (≤ 60 Minutes)

Step	Time	Outcome
Bootstrap environment	5m	SDKs & Dapr installed
Run template service	5m	Local API up
Execute tests	10m	Green tests & coverage
Add endpoint + unit test	15m	Pattern comprehension
Add contract test & publish	10m	Pact in broker
Build & scan container	10m	Security integrated
Review pipeline YAML	5m	Understand gates

19. Drift Management

Quarterly automation:

- Compare active services vs template manifest (hash of canonical file list + baseline).
- Generate drift report: missing analyzers, outdated Dockerfile base digest, absent OTel exporter.
- Create remediation tickets (SLA: 1 sprint for critical drift, 2 sprints for minor).

20. Risks & Mitigations

Risk	Impact	Mitigation
Tool Sprawl	Inconsistent builds	Central catalog + ADR approval
Skipped Contract Tests	Runtime incompatibility	Pipeline hard gate
Over-reliance on Defaults	Hidden security gaps	Security review checklist
Performance Blindness	Latency regressions	Mandatory performance smoke stage
Template Divergence	Increased maintenance	Drift audit & auto PR suggestions

21. Backlog (Related Stories)

Ref	Title	Priority
DEVTOOL-001	Publish k12-cli template package	High
DEVTOOL-002	Implement custom idempotency analyzer	High
DEVTOOL-003	Add integrityHash generator CLI utility	High
DEVTOOL-004	Pact broker container + seed example	High

DEVTOOL-005	Dapr local profile script (multi-service)	Medium
DEVTOOL-006	Template drift auditor job	Medium
DEVTOOL-007	Coverage trend dashboard	Medium
DEVTOOL-008	Security waiver tracking automation	Medium

22. Acceptance Criteria (Representative)

- Creating new service via k12-cli new service yields compiling solution with passing unit tests, coverage $\geq 70\%$.
- Contract test failing on provider side blocks merge.
- Pushing code with an embedded secret (test string) fails CI.
- Running local template produces OpenAPI doc at /swagger and /health endpoints reachable.
- integrityHash CLI tool produces identical hash for same payload across dev machines.

23. Future Enhancements (Phase 2+)

Enhancement	Rationale
Tail-based sampling (OTel)	Reduce trace cost while preserving anomalies
Policy as Code (OPA)	Enforce Terraform & container policies
Native AOT edge services	Reduced cold start & memory
AsyncAPI documentation for events	Improve event contract clarity
Automatic golden-path generator dashboard	Visual onboarding status

24. Compliance Mapping (Selected)

Requirement	Implementation
-------------	----------------

Auditability	Git history + signed image digest (Phase 2)
Least Privilege	Service principal / Managed Identity per service
Data in Transit Encryption	TLS enforced; no HTTP fallback
No Secrets in Code	Gitleaks + pre-commit + reviews
Traceability	CorrelationId mandated in middleware

25. Change Management

- Minor version increments: additions or clarifications (no breaking structure).
- Major version increments: structural changes to template, new mandatory tools, or coverage threshold increase.
- All major changes require Architecture Board approval + broadcast.

26. Readiness Checklist (Per Service Before First Deploy)

Item	Pass
Uses standard template structure	☐
Health endpoints implemented	☐
OpenTelemetry enabled (traces + RED metrics)	☐
Contract tests published & validated	☐
Coverage $\geq$ threshold	☐
Dockerfile matches approved base digest	☐
Trivy scan clean (no Critical)	☐
integrityHash enforced in message publisher (if applicable)	☐
Idempotent consumer attribute applied (if applicable)	☐
No secrets detected by Gitleaks	☐

Testing

Load testing Azure microservices typically involves using Azure Load Testing, a fully managed load-testing service that allows you to generate high-scale load and analyze application performance.

- **Create an Azure Load Testing Resource:**
 - Create a new Azure Load Testing resource, providing details like subscription, resource group, name, and region.
- **Create a Load Test:**
- **URL-based test:** If you have a simple scenario and want to test a single URL, you can define the request details directly in the portal.
 -
 - **Script-based test:** For more complex scenarios involving multiple requests, authentication, or dynamic data, you can upload a test script. Azure Load Testing supports Apache JMeter and Locust scripts.
- **Configure the Load Test:**
 - **Load Pattern:** Define the number of virtual users, ramp-up period, and test duration to simulate the desired load.
 - **Target Endpoints:** Specify the URLs or endpoints of your Azure microservices that you want to test.
 - **Parameters and Secrets:** If your microservices require specific parameters, headers, or secrets (e.g., API keys), you can configure them in your script or directly in the Azure Load Testing settings.
 - **Failure Criteria:** Define conditions under which the test should be considered a failure (e.g., high error rate, slow response times).
- **Run the Load Test:**

- Initiate the load test from the Azure portal.
- Azure Load Testing will provision the necessary test engines and execute your test script, generating load against your microservices.
- **Analyze Results:**
 - During and after the test run, you can monitor real-time metrics and view detailed results in the Azure Load Testing dashboard.
 - Analyze key performance indicators (KPIs) like response time, throughput, error rate, and resource utilization of your microservices
 - Identify bottlenecks, performance regressions, and areas for optimization within your microservice architecture.
- **Iterate and Optimize:**
 - Based on the analysis, make necessary adjustments to your microservices for scaling, code optimization, configuration .
 - Rinse and Repeat the load testing process

Stress Testing

A performance test that overloads a system until it breaks. Push your workload beyond its normal limits to identify its breaking points and measure its ability to recover. SEAA/CFI has given concurrency and user count for us to baseline for testing. Sensei should minimally be able to handle spikes of heavy activity.

<https://learn.microsoft.com/en-us/azure/well-architected/performance-efficiency/performance-test>

<https://azure.microsoft.com/en-us/products/load-testing>

Load Testing

Load testing is critical for confirming that your application can handle expected user traffic by simulating real-world demand. Sensei should minimally be able to handle spikes of heavy activity.

<https://learn.microsoft.com/en-us/azure/well-architected/performance-efficiency/performance-test>

<https://azure.microsoft.com/en-us/products/load-testing>